Project Report:
Automated Data Cleaning & Recommendation System

....

June 14, 2026

Contents

1 Project Assumptions

The main objective of the project was to design and implement an automated system for retrieving, verifying, and cleaning weather data from anomalies occurring within them.

The following assumptions were made as part of the project:

- Retrieving raw time-series data from an external, public weather API.
- Implementing a multi-tier data storage architecture, dividing datasets into four structures: *Bronze* (raw JSON data), *Quarantine* (completely rejected data), *Silver* (data cleaned of errors), and *Gold* (audit logs of completed operations).
- Implementing anomaly detection algorithms using the statistical measure *Z-score*.
- Automatic repair of anomalies by replacing them with a rolling mean calculated from the last 14 measurements.
- Creating an intuitive user interface (UI) to ensure system operation transparency. The interface is intended to allow the selection of weather stations, data preview at each processing stage (raw and cleaned), and a review of system logs.

2 Technologies and Libraries Used

The project was implemented in the **Python** language using the following libraries and tools:

- **Streamlit** – for building an interactive user interface (UI).
- **Pandas** – for manipulation, aggregation, and processing of datasets in the form of dataframes.
- **NumPy** – for optimizing numerical calculations.
- **Requests** – for handling the HTTP protocol and communication with the external weather API.
- **JSON** – for serialization and deserialization of raw data and configuration files.
- **Python-dotenv** – for secure management of environment variables (tokens, URLs).
- **uv** – as a modern and efficient package and virtual environment manager.

3 Architecture and Data Flow

1. **Bronze Layer:** The `ingestion.py` script saves raw JSON objects here, exactly in the form they were returned by the API.
2. **Quarantine:** Records that failed to meet the assumptions defined in `cleaning_rules.json` and are too mismatched to undergo repair end up here.
3. **Silver Layer:** Data that passed initial validation are checked for soft anomalies (Z-score). Detected deviations are substituted, and the clean dataset is ready for analysis.

4. **Gold Layer:** The event register. It documents every intervention of the algorithm into the original measurement value.

4 Project Structure

The project tree has been divided to ensure separation of business logic, data layer, and user interface.

```
automated-data-cleaning-system/
|-- config/                  # Configuration folder
|   |-- .env                 # Hidden environment variables (URL, token)
|   |-- cleaning_rules.json  # Dictionary of strict validation rules
|   \-- utils.py             # Helper functions and constants
|-- notebooks/               # Jupyter environment for testing and EDA
|   \-- final_report.ipynb
|-- src/                     # Main source code of the system
|   |-- data/                # Local file repository
|   |   |-- bronze/          # Raw data from API (divided by stations)
|   |   |-- quarantine/      # Rejected records
|   |   |-- silver/          # Processed data
|   |   \-- gold/            # Repair history (Audit logs)
|   |-- expert_system/       # Rule logic (Orchestration)
|   |   \-- decision_engine.py
|   |-- models/              # Analytical logic
|   |   |-- baseline_stats.py   # Statistical module (Z-score)
|   |   \-- imputation_ml.py    # Data imputation algorithms
|   |-- pipeline/            # Extraction and Input/Output operations
|   |   |-- audit_loger.py
|   |   |-- ingestion.py
|   |   \-- storage_manager.py
|   \-- ui/                  # User interface (Frontend)
|       |-- constants.py
|       \-- home.py
|-- video/                   # Demonstration materials
|   \-- video.mp4
|-- main.py                  # Application initializing script
|-- pyproject.toml           # Project metadata and configuration
|-- uv.lock                  # UV manager dependency lockfile
\-- README.md
```

5 Code Review

The focus was placed on key system functions responsible for processing and cleaning data. An analysis of the most important code fragments is presented below.

5.1 Statistical Anomaly Detection (`zScoreDecisionMaking`)

This function constitutes the analytical core of the Silver layer, responsible for identifying so-called "soft" anomalies based on standard deviation.

```
def zScoreDecisionMaking(data: pd.DataFrame):
    data.sort_values(by=['station_id', 'timestamp'], inplace=True)
    rules = load_rules()
    for FEATURE in FEATURES_TO_CHECK:
```

```
        if FEATURE in ['rain_mm', 'wind_direction']:
            # Logic for non-standard features
            pass
        else:
            data[f'{FEATURE}_rolling_mean'] = data.groupby('station_id')[f'{FEATURE}'
    ↪ ]\
                .transform(lambda x: x.rolling(window=14, min_periods=1).mean())
            data[f'{FEATURE}_rolling_std'] = data.groupby('station_id')[f'{FEATURE}']\
                .transform(lambda x: x.rolling(window=14, min_periods=1).std())

            data[f'{FEATURE}_z_score'] = (data[f'{FEATURE}'] - data[f'{FEATURE}
    ↪ _rolling_mean']) \
                / data[f'{FEATURE}_rolling_std']

            data[f'{FEATURE}_is_anomaly'] = (data[f'{FEATURE}_z_score'].abs() >
    ↪ MEAN_MULTIPLE) \
                & (data[f'{FEATURE}_rolling_std'].abs() > 0)
    return data
```

Instead of iterating over each row, vector functions of the Pandas library (`groupby`, `transform`, `rolling`) were utilized, which guarantees high computational performance. Application of the `min_periods=1` parameter allows for maintaining computational continuity even at the beginning of the dataset.

5.2 Data Imputation and Logging (`fixAnomalies`)

A function that repairs corrupted rows and generates entries for the log journal.

```
def fixAnomalies(data: pd.DataFrame):
    changes = []
    for feature in FEATURES_TO_CHECK:
       if f'{feature}_is_anomaly' in data.columns:
            feature_anomaly = data[data[f'{feature}_is_anomaly'] == True]
            for _, row in feature_anomaly.iterrows():
                changes.append({
                    'timestamp': row['timestamp'],
                    'station_id': row['station_id'],
                    'feature': feature,
                    'original_value': row[feature],
                    'new_value': row[f'{feature}_rolling_mean'],
                    'action': 'Z-Score␣Imputation'
                })

            # Direct substitution of corrupted data
            data[feature] = np.where(data[f'{feature}_is_anomaly'] == True,
                                     data[f'{feature}_rolling_mean'],
                                     data[feature])
    return data, changes
```

The use of the `numpy.where()` function for conditional value overwriting. This works instantly across entire columns.

5.3 Hard Rules Validation (`checkRules`)

A function rejecting physically impossible measurements (e.g., temperature below absolute zero).

```
def checkRules(data: pd.DataFrame):
    rules = load_rules()
    data['is_anomaly'] = False

    for FEATURE in FEATURES_TO_CHECK:
        # Bitwise OR operator for consolidating errors from multiple columns
        data[f'is_anomaly'] |= (data[f'{FEATURE}'] < rules[f'{FEATURE}']['min']) \
                             | (data[f'{FEATURE}'] > rules[f'{FEATURE}']['max'])

    clean_data = data[~data['is_anomaly']]
    quarantine_data = data[data['is_anomaly']]
    return clean_data, quarantine_data
```

Using the bitwise `|=` operator allows for gathering error information from all columns into a single collective `is_anomaly` flag. Next, the code creates two independent datasets (clean and quarantined) using the negation `~`.

5.4 Final Data Cleaning (`data_cleaning`)

```
def data_cleaning(df: pd.DataFrame):
    if df is None:
        df = createDataFrame()

    clean_data, quarantine_data = checkRules(df)
    processed_df = zScoreDecisionMaking(clean_data)
    fixed_Data, change_logs = fixAnomalies(processed_df)
    cleaned_data = cleanData(fixed_Data)

    # Separation and storage of data into appropriate layers
    exportData(quarantine_data, 'src/data/quarantine', 'quarantine_data.json')
    exportData(cleaned_data, 'src/data/silver', 'silver_data.json')

    change_logs_df = pd.DataFrame(change_logs)
    save_audit_logs(change_logs_df, 'audit_logs.json')

    return cleaned_data
```

Appropriate modules are loaded in a specified order (Loading → Hard Rules → Z-Score → Imputation → Saving).

6 Discussion of Achieved Results

The following section focuses on discussing the achieved outcomes.

6.1 Advantages of the Solution

- Creation of a user-friendly interface (UI) that effectively visualizes complex background processes and facilitates program usage.
- Possibility of parallel data retrieval and processing from different weather stations in a single cycle.

- Division into four independent clusters protects original data from overwriting and degradation.
- Systematic addition of logs guarantees transparency of the process and facilitates subsequent debugging of the statistical algorithm's behavior.

6.2 Disadvantages, Limitations, and Development Opportunities

- The user interface is currently quite simple, and the cleaning process is fully automatic. It could be expanded by giving the user the opportunity to decide on detected anomalies and manually approve repair actions.
- Currently, anomalies are patched solely based on the rolling mean of the last 14 measurements. In the future, a regression model using machine learning could be created to predict the missing value by taking into account correlations with other variables (e.g., pressure or wind force), which would increase completion precision.
- The system is limited by the single packet download threshold from the API (the limit is around 1000 records due to provider server settings).